

07/17/25 - Video 1.11 - Best, Worst, and Average Case Analysis

We'll be focusing on linear search and binary search tree.

Linear Search

A	8	6	12	5	9	7	4	3	16	18
	0	1	2	3	4	5	6	7	8	9

→ So how does linear search work? Say we're looking for the value 7. This is referred to us as a key. We start from the left and then go to the right. Not an index 0, 1, ... Bam at index 5. This then takes 6 steps.

Now what if we're looking for 93. Again, start all the way from the left and go right. Doesn't exist. So it's a waste of time.

Best Case/Time

the key element we're searching for is present at the first index, index 0. The best case time is $O(1)$.

Worst Case/Time

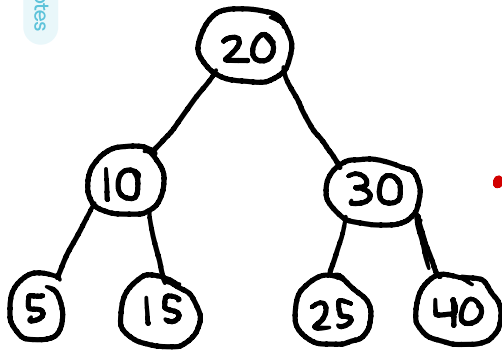
the key element we're searching for is present at the last index, index 9 here. The worst case time is $O(n)$.

Average Case/Time

this is defined by
$$\frac{\text{all possible case times}}{\text{\# of cases}}$$

This is hard and isn't possible on every algorithm, so we rarely try to find this. And usually, it same as worst case.

Binary Search Tree



- the way this bst is set up is that the smallest element is on the left and the largest element is on the right.
- Always start at the root and begin searching from there.

* the height of a tree, we classify it as $\log n$ *

Best Case/Time

the element we're searching for is at the root. $O(1)$.

Worst Case/Time

the element we're searching for is at the leaf. $O(\log n)$.

Let's say in a hypothetical scenario that our BST is left skewed, meaning its . The best case and worst case is still the same. **HOWEVER**, the worst case can vary. In this example, its height is n b/c every node has a left child. So it's lined up like a linked list. $\log n$ occurs when we roughly have the number of nodes in the left and right trees balanced.

7/17/25 - Video 1.11 Continued

The minimum worst case time is $O(\log n)$.

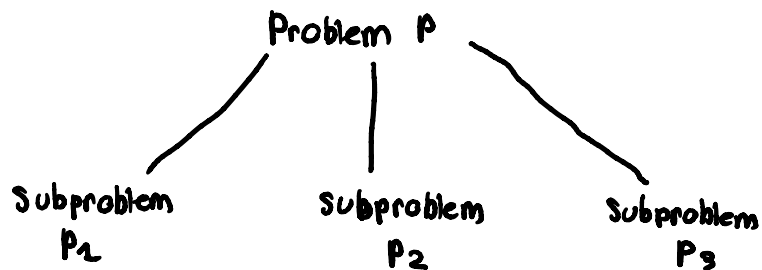
The maximum worst case time is $O(n)$.

A bst can have a height of $\log n$, this is the one that we often see with nodes, children, leaves. It's n when it's just a one sided tree, so either left sided or right sided.

When the minimum worst case = $\log n$, this is the best scenario among all the worst cases. This means the tree is balanced

When the maximum worst case = n , this is the worst scenario among all the worst cases. This means the tree is skewed. And this is the maximum possible height. B/c if we have  go on, it takes a long time to search, especially if it's in the lowest node.

Divide and Conquer is a strategy we use to solve a problem. We do exactly as the name suggests. We will break up a problem into smaller parts, solve them each, and then join them all back together.



We individually solve each problem, then combine the solutions to solve the main problem.

Divide and conquer is a recursive problem.

- Some common divide and conquer are
1. binary search
 2. Finding max and min
 3. mergesort
 4. quicksort
 5. matrix multiplication